VitalFlow — UI Guidelines

> Shared design system + app-shell conventions for the unified web app. Provider, admin, and patient surfaces live as route groups under a single deployment. One system, one taxonomy, one set of a11y rules.

1. Component taxonomy

Four layers, each with a clear role and blast radius. **Components only depend on layers below them.**

Layer	Path	Purpose	Examples
Tokens	packages/ui/src/styles/tokens.css	CSS custom properties (colors, radii, motion, geometry). Tenant-themable at runtime.	`--vf-primary` , `--vf-radius` , `--vf-sidebar-width`
Primitives	packages/ui/src/primitives/	Generic, single-purpose, domain-agnostic. Thin wrappers around Radix + Tailwind.	`Button` , `Input` , `Select` , `Textarea` , `Badge` , `Card` , `Modal` , `Drawer` , `Table` , `Tabs` , `Label` , `Separator` , `FormField`
Patterns	packages/ui/src/patterns/	Composed behaviors reused across pages. Stateless, data-shape-agnostic.	`PageHeader` , `EmptyState` , `ErrorState` , `LoadingState` , `Skeleton` , `DataTable`
Layout	packages/ui/src/layout/	App-shell pieces. Accept slots/manifests so each app composes without the package knowing the app.	`SidebarLayout` , `Sidebar` , `TopNav` , `AppBreadcrumbs` , `AuthGuard`
Clinical	packages/ui/src/clinical/	Domain composites (PHI-aware).	`VitalsCard` , `AllergyBadge` , `EncounterTimeline` (populated per milestone)

Hard rules:

- Primitives **never** import from patterns/layout/clinical.
- Patterns **never** import from layout/clinical.
- No component imports from an app — UI flows one way: `ui` → `app` .
- If a component would need tenant/user context to render, it takes that as a prop, not from a global.

2. Design token structure

Token names are CSS custom properties prefixed `--vf-` (scoped to avoid collisions with third-party libraries). Categories:

Group	Tokens
Surface	`background` , `foreground` , `border` , `input` , `ring`
Brand	`primary` , `primary-foreground` , `secondary` , `secondary-foreground`
Neutral	`muted` , `muted-foreground` , `accent` , `accent-foreground`

Status	`destructive`, `success`, `warning`, `info` (+ `-foreground` pair for each)
Clinical	`clinical-critical`, `clinical-warning`, `clinical-normal`, `clinical-info`
Geometry	`radius`, `sidebar-width`, `sidebar-width-collapsed`, `topnav-height`
Type	`font-sans`, `font-mono`
Motion	`duration-fast`, `duration-normal`, `duration-slow`, `ease`
Elevation	`shadow-sm`, `shadow-md`, `shadow-lg`

Theme switching: add the `.dark` class to `<html>`; all tokens have dark-mode counterparts.

Tenant branding: override a subset at runtime by injecting a `<style>` tag that redeclares `:root { --vf-primary: … }`. Tailwind classes keep working unchanged.

3. `packages/ui` directory structure

```
packages/ui/
├── src/
│   ├── primitives/      # Atomic components (1 file per primitive)
│   ├── patterns/        # Reusable composed behaviors
│   ├── layout/          # App shell (sidebar, topnav, auth-guard, breadcrumbs)
│   ├── clinical/        # Domain composites (populated per milestone)
│   ├── icons/           # Curated lucide-react re-exports
│   ├── styles/
│   │   ├── tokens.css   # CSS custom properties
│   │   └── index.css    # Tailwind base + token wiring
│   ├── utils/
│   │   └── cn.ts        # clsx + tailwind-merge
│   └── index.ts         # Barrel: re-exports everything
├── package.json         # Subpath exports: ./primitives, ./patterns, ./layout, ./icons, ./styles, ./tokens
└── tsconfig.json
```

Subpath imports let apps tree-shake:

```
import { Button, Card } from "@vitalflow/ui";            // barrel
import { DataTable } from "@vitalflow/ui/patterns";      // narrower
import { SidebarLayout } from "@vitalflow/ui/layout";
import { Stethoscope } from "@vitalflow/ui/icons";
```

4. Sample base components

Implemented with consistent API shape. All source paths are clickable.

Component	Where	Notes
Button	Radix `Slot` + CVA variants (default, outline, ghost, destructive, link, secondary)	`asChild` pattern for composing into `<Link>` or `<a>`

Input	Native `<input>` with `invalid` → `aria-invalid` wiring	Extends `InputHTMLAttributes`
Select	Radix `@radix-ui/react-select` dressed with tokens	Scroll buttons, keyboard nav, portal content
Textarea	Native `<textarea>` with same `invalid` contract	
Badge	CVA variants incl. `clinical-*` semantic colors	Use `muted` for neutral counts
Card	`Card` + `CardHeader` + `CardTitle` + `CardContent` + `CardFooter`	Default shadow `shadow-vf-sm`
Modal	Radix Dialog, center-positioned	`Modal`, `ModalContent`, `ModalHeader`, `ModalTitle`, `ModalDescription`, `ModalFooter`, `ModalClose`
Drawer	Radix Dialog, side-positioned (`left`, `right`, `top`, `bottom`)	Powers the mobile nav in `SidebarLayout`
Table	Styled HTML table wrappers (`Table`, `TableHeader`, `TableBody`, `TableRow`, `TableHead`, `TableCell`, `TableCaption`, `TableFooter`)	Prefer the DataTable pattern for list pages.
Tabs	Radix Tabs	
FormField	Label + control + helper/error, with auto-wired `id` / `aria-describedby`	**Always** wrap form inputs in this; never hand-roll label spacing.

Patterns

Pattern	Use for
PageHeader	Title + eyebrow + description + actions. Top of every page.
EmptyState	Zero-data state inside cards, tables, tabs. Must include a primary action when one exists.
ErrorState	Recoverable error surface. `technical` prop collapses a stack trace for devs.
LoadingState / Skeleton	Prefer `Skeleton` blocks when layout size is known; fall back to `LoadingState` spinner.
DataTable	List pages. Handles `idle/loading/empty/error` states in one component.

Shell

Component	Purpose
SidebarLayout	Canonical app shell. Sidebar slot, TopNav slot, main content. Mobile drawer built-in.
Sidebar	Renders a `NavSection[]` manifest. Filters by permissions. Link component injected by host.
TopNav	Sticky header with mobile hamburger, breadcrumb slot, action slot, user dropdown.
AppBreadcrumbs	Accessible breadcrumb trail (`aria-current="page"` on last item).
AuthGuard	UX guard (loading / unauthenticated / forbidden states). **RLS in Postgres is the real boundary.**

5. App shell layout

The unified web app uses `SidebarLayout`. The active surface (provider / admin / patient) is derived from the URL, and the shell picks the matching nav manifest. The app injects:

- **Brand** — a surface-aware logo slot with a small "· Admin" / "· My health" caption when off the provider surface
- **NavSection[]** — one of three manifests (provider, admin, patient), selected by `navFor(surface)`
- **NavContext** — `{ permissions, pathname }`; pathname from `usePathname()`, permissions from `@vitalflow/auth`
- **LinkComponent** — Next.js `Link` (keeps `packages/ui` framework-agnostic)
- **User** — `{ name, email?, avatarUrl?, roleLabel? }` for the top-right dropdown
- **SurfaceSwitcher** — pill group shown when a user carries roles for more than one surface

Each `NavItem` uses `requires: string[]` for permission-gated items and `comingSoon: true` for stubs.

Route structure:

```
apps/web/src/app/
├── layout.tsx                  # root: html + body + tokens.css
├── (auth)/                     # public surfaces
│   ├── layout.tsx              # minimal, no sidebar
│   └── login/page.tsx          # /login
└── (app)/                      # authenticated shell
    ├── layout.tsx              # resolves session + AppShell
    ├── _shell/app-shell.tsx    # SidebarLayout (client) — picks nav by URL
    ├── page.tsx                # /          (provider dashboard, default)
    ├── admin/
    │   ├── layout.tsx          # gate: requires admin surface
    │   └── page.tsx            # /admin
    └── my/
        ├── layout.tsx          # gate: requires patient surface
        └── page.tsx            # /my
```

Surface resolution — the shell reads `usePathname()` and calls `surfaceForPath()` to pick `provider` / `admin` / `patient`. This keeps the URL as the single source of truth — no cookie drift between what the user sees and where they are.

6. Coding standards

Component shape

```
"use client"; // only when needed (hooks, Radix, event handlers, effects)

import * as React from "react";
import { cn } from "../utils/cn.js";

export interface FooProps extends React.HTMLAttributes<HTMLDivElement> {
  variant?: "default" | "destructive";
}

export const Foo = React.forwardRef<HTMLDivElement, FooProps>(
  ({ className, variant = "default", ...props }, ref) => (
    <div ref={ref} className={cn(/* ... */, className)} {...props} />
  ),
);
Foo.displayName = "Foo";
```

Rules:

1. `forwardRef` for everything rendering a DOM element.
2. `displayName` set on every `forwardRef` component.
3. Extend the native HTML element's props type — never reinvent `onClick` / `className` /etc.
4. `className` prop always composed last through `cn()` so callers can override.
5. Variants via `class-variance-authority`. Never branch with ternaries inside JSX for style.
6. `"use client"` **only** when the component uses hooks, Radix primitives, or event handlers. Card/Badge/Input are safe in Server Components.
7. All imports use explicit `.js` extensions (ESM + `verbatimModuleSyntax`).

Accessibility

- Every interactive element is reachable by keyboard and has a visible `focus-visible:ring-2 ring-ring ring-offset-2`.
- Icons inside buttons get `aria-hidden` and the button gets `aria-label` (or visible text).
- Use `role="status"` / `role="alert"` / `aria-live` for dynamic state regions (we bake these into `EmptyState`, `LoadingState`, `ErrorState`).
- Form controls always paired with `<Label htmlFor>` — use `FormField` to avoid forgetting.
- Errors connected to inputs via `aria-invalid` and `aria-describedby`.
- Never use color alone to convey state (pair with icon or text).

Imports

```
import * as React from "react";        // React first
import { Foo } from "@radix-ui/...";   // third-party
import { Button } from "@vitalflow/ui"; // monorepo
import { localThing } from "./local";   // relative last
```

ESLint enforces this via `import/order`.

Styling

- Tailwind classes only. No inline `style` except for computed values (e.g. dynamic transforms).
- No hard-coded colors. Every color lands in tokens.
- Use the design-token Tailwind utilities (`bg-primary` , `text-muted-foreground`) — never raw hex.
- Motion uses `duration-fast/normal/slow` + `ease-vf` utilities.

Client vs server

- Default to Server Components.
- Mark `"use client"` only when needed. Patterns like `PageHeader` , `EmptyState` , `Card` , `Badge` , `Input` are server-safe.
- Shell components (`SidebarLayout` , `Sidebar` , `TopNav` , `AuthGuard`) are client because they manage interaction.

Testing (future)

- Component tests: Vitest + React Testing Library, run via `pnpm --filter @vitalflow/ui test` .
- Visual regression: Chromatic or Playwright snapshot — TBD when the first theme fork ships.
- Accessibility: axe checks integrated in Playwright E2E.

Quick reference — starting a new page

```
// apps/web/src/app/(app)/some-feature/page.tsx
import { Button, Card, CardContent, CardHeader, CardTitle } from "@vitalflow/ui";
import { AppBreadcrumbs } from "@vitalflow/ui/layout";
import { DataTable, PageHeader } from "@vitalflow/ui/patterns";
import NextLink from "next/link";

export default function Page() {
  return (
    <>
      <AppBreadcrumbs
        LinkComponent={NextLink}
        items={[{ label: "Home", href: "/" }, { label: "Some feature" }]}
      />
      <PageHeader title="Some feature" description="Why this page exists." actions={<Button>Primary</B
      {/* DataTable or Cards here */}
    </>
  );
}
```